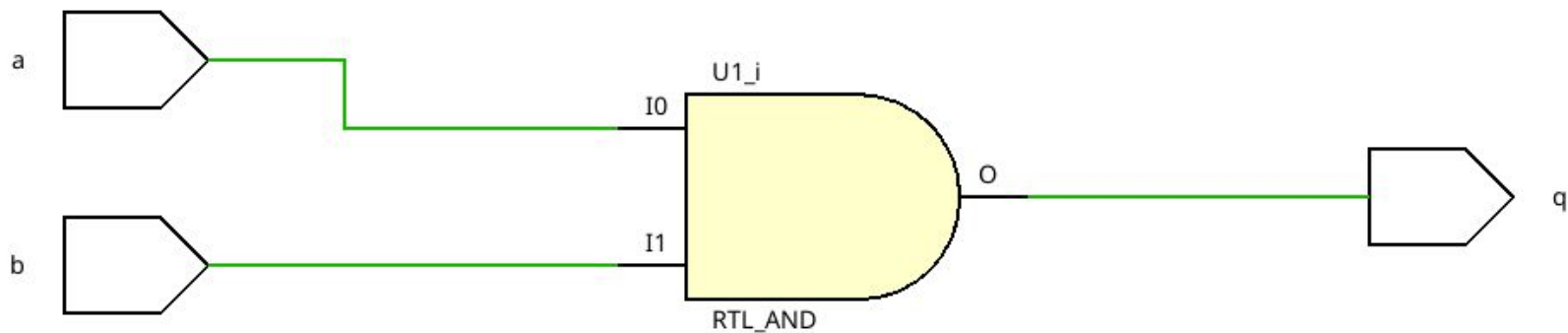


AND Gate (Schematic)



AND Gate (Verilog)

// [동작적] 방식 : 동작을 코드로 표현 / 복잡한 로직 처리에 유리

```
module and_gate_behavioral(  
    input a,b,           // 입력 신호 a, b  
    output reg q         // 출력 신호 q (레지스터 타입)  
);  
    always @(a or b) begin                // a 또는 b 신호가 변경될 때 실행  
        if (a == 1'b1 && b == 1'b1) begin // a와 b가 모두 1일 때  
            q = 1'b1;                     // 출력 q를 1로 설정  
        end else                          // 그렇지 않을 때  
            q = 1'b0;                     // 출력 q를 0으로 설정  
        end  
    end  
endmodule
```

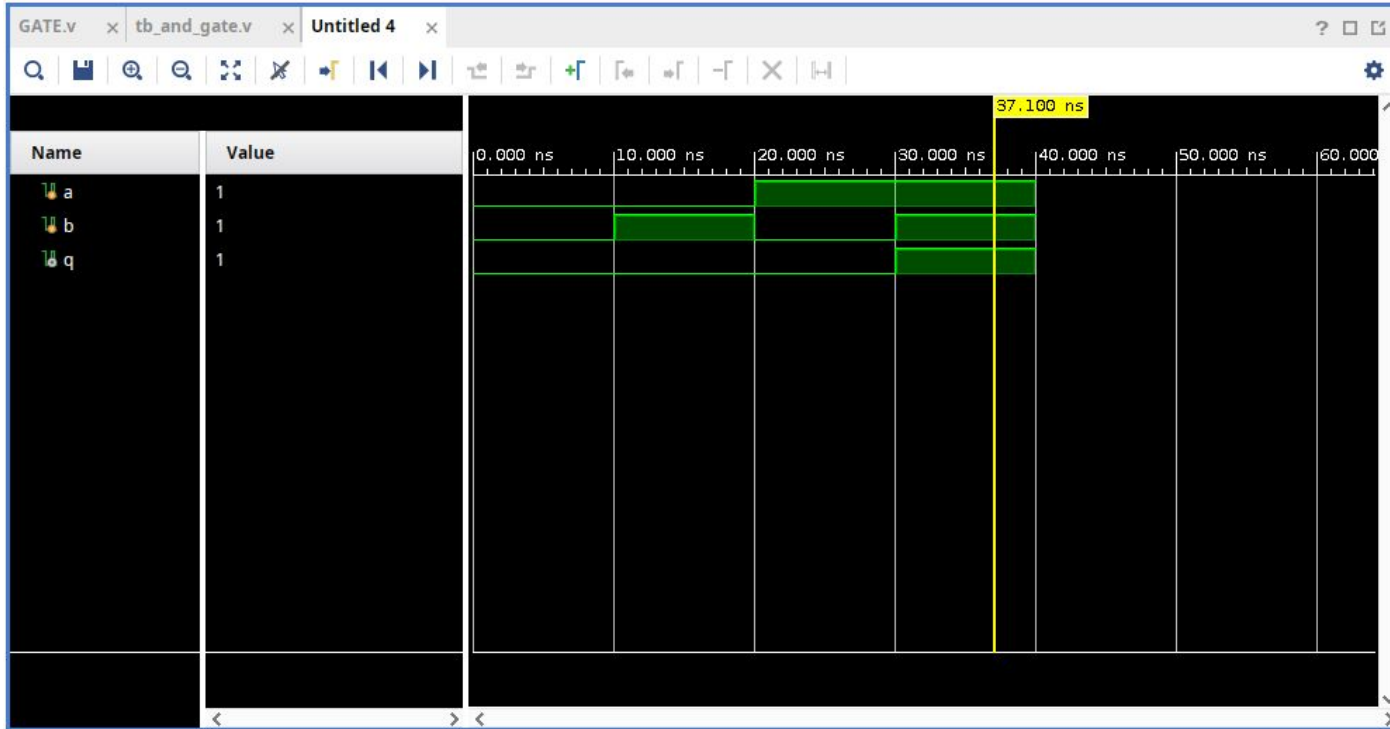
// [구조적] 방식 : 실제 게이트(AND) 이용해서 회로의 구조를 기술 하드웨어의 구성요소를 직접 인스턴스 화함.

```
module and_gate_structural (  
    input a, b,  
    output q // 여기 주의 !!! wire 타입이어야 함.  
);  
    and U1(q, a, b);  
endmodule
```

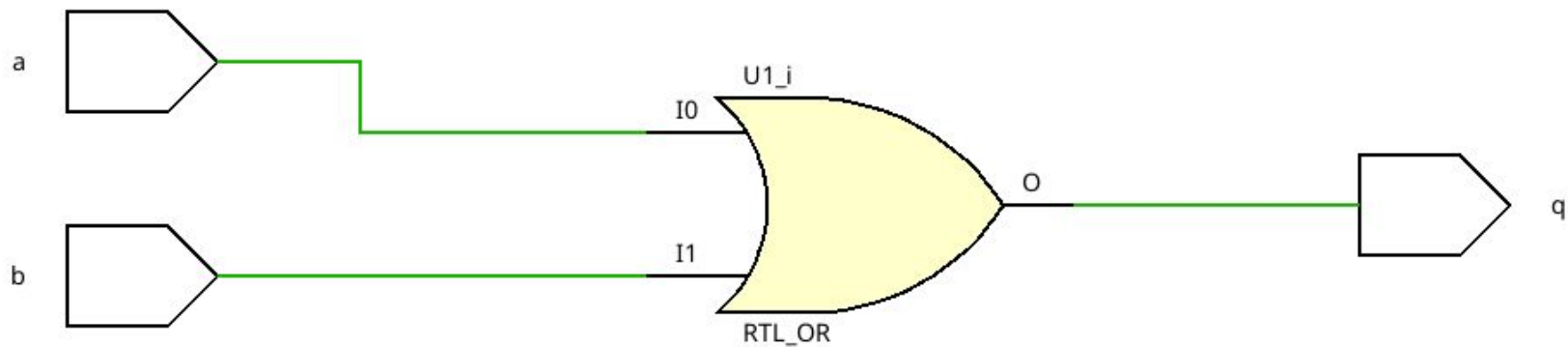
// [데이터플로우] 방식 : 데이터의 흐름 중심 assign 문으로 출력과 입력간의 논리를 기술

```
module and_gate_dataflow (  
    input a, b,  
    output q // 여기 주의 !!! wire 타입이어야 함.  
);  
    assign q = a & b;  
endmodule
```

AND Gate (Simulation)



OR Gate (Schematic)



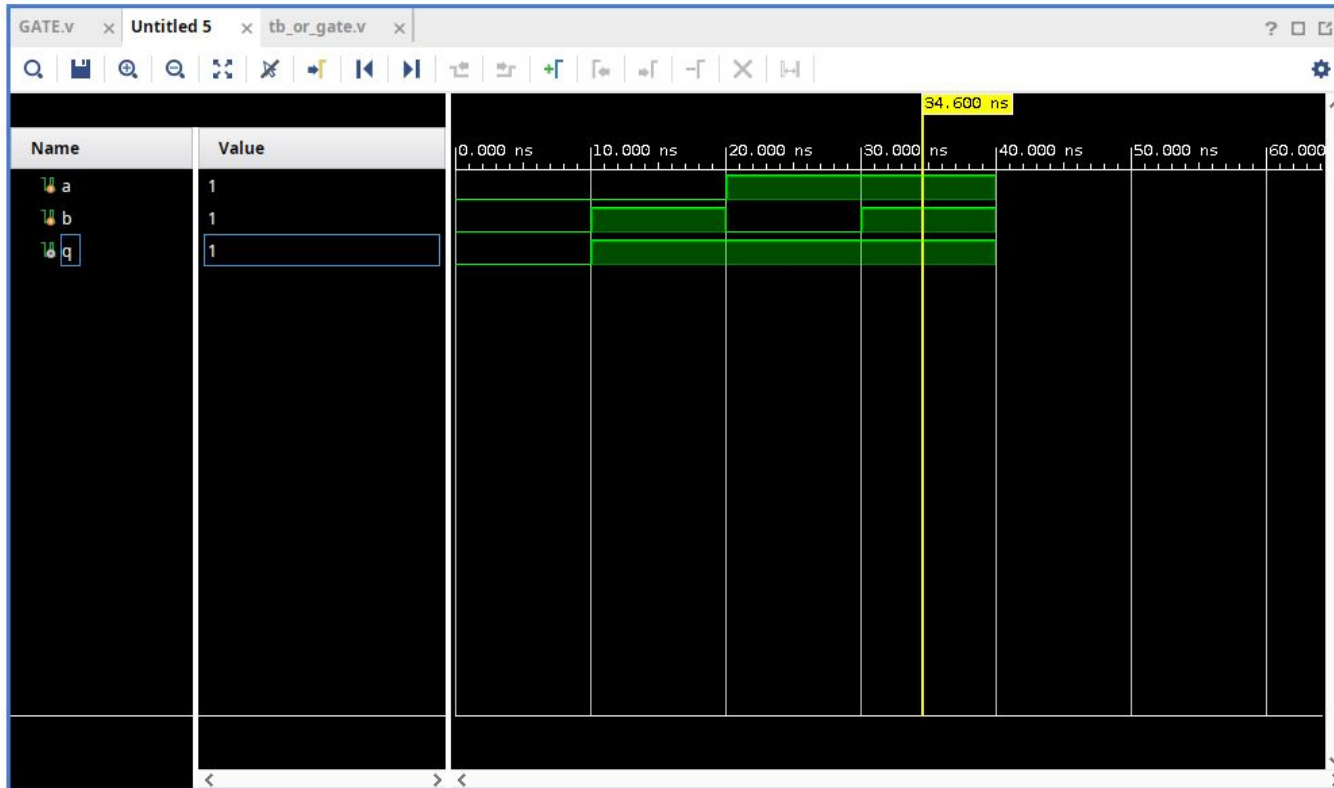
OR Gate (Verilog)

```
module or_gate_behavioral (  
    input a, b,  
    output reg q);  
  
    always @(a,b) begin  
        if (a == 1'b1 || b == 1'b1)  
            q = 1'b1;  
        else  
            q = 1'b0;  
        end  
    end  
endmodule
```

```
module or_gate_structural(  
    input a, b,  
    output q);  
  
    or U1(q, a, b);  
  
endmodule
```

```
module or_gate_dataflow (  
    input a, b,  
    output q // 여기 주의 !!! wire 타입이어야 함.  
);  
    assign q = a | b;  
endmodule
```

OR Gate (Simulation)



```
# run 1000ns
Time    a b | q
0      0 0 | 0
10000   0 1 | 1
20000   1 0 | 1
30000   1 1 | 1
```

NAND Gate (Schematic)



NAND Gate (Verilog)

```
module nand_gate_behavioral (
    input a, b,           // 입력 신호 a, b
    output reg q          // 출력 신호 q (레지스터 타입)
);
    always @(a,b) begin // a 또는 b 신호가 변경될 때 실행
        if (a == 1'b1 && b == 1'b1) // 둘 다 1일 때
            q = 1'b0;              // 출력 0 (NAND는 둘 다 1일 때만 0)
        else                      // 하나라도 0이면
            q = 1'b1;              // 출력 1
        end
    end
endmodule
```

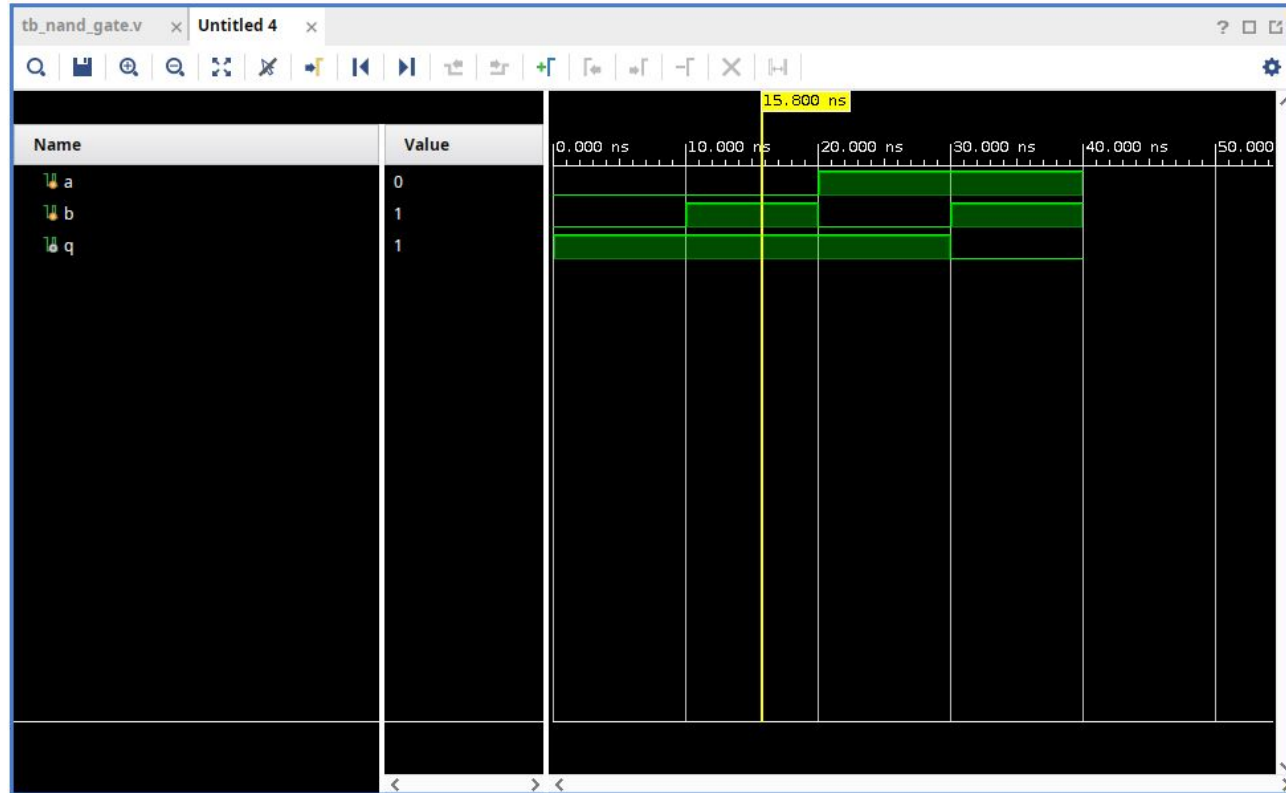
```
module nand_gate_structural(
    input a, b,           // 입력 신호 a, b
    output q              // 출력 신호 q (wire 타입)
);

    nand U1(q, a, b);    // 내장 NAND 게이트 인스턴스 생성

endmodule
```

```
module nand_gate_dataflow (
    input a, b,           // 입력 신호 a, b
    output q              // 출력 신호 q (wire 타입)
);
    assign q = ~(a & b); // AND 연산 후 반전하여 NAND 구현
endmodule
```

NAND Gate (Simulation)



```
# run 1000ns
```

```
Time      a b | q
          0   0 0 | 1
        10000 0 1 | 1
        20000 1 0 | 1
        30000 1 1 | 0
```

NOR Gate (Schematic)



NOR Gate (Verilog)

```
module nor_gate_behavioral (
    input a, b,          // 입력 신호 a, b
    output reg q          // 출력 신호 q (레지스터 타입)
);
    always @(a,b) begin // a 또는 b 신호가 변경될 때 실행
        if (a == 1'b0 && b == 1'b0) // 둘 다 0일 때만
            q = 1'b1; // 출력 1 (NOR 게이트는 모두 0일 때만 1)
        else // 하나라도 1이면
            q = 1'b0; // 출력 0
        end
    end
endmodule
```

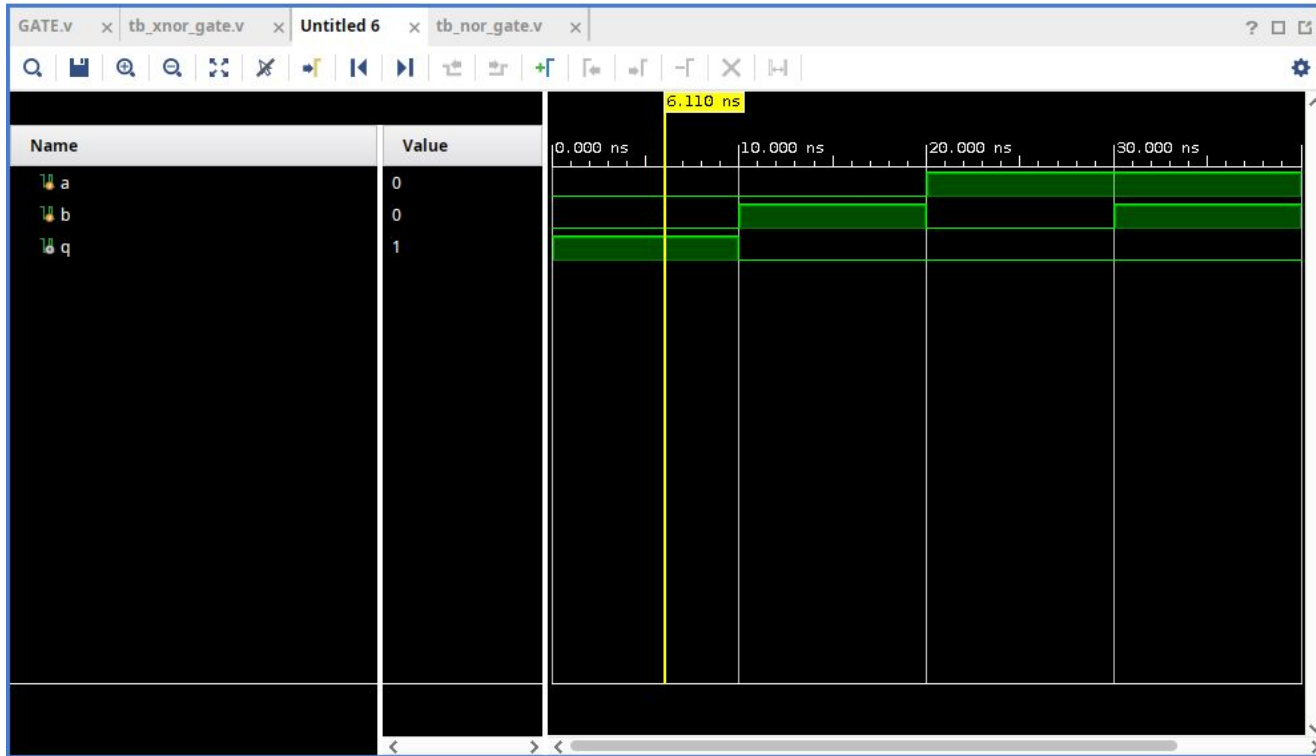
```
module nor_gate_structural(
    input a, b,          // 입력 신호 a, b
    output q              // 출력 신호 q (wire 타입)
);

    nor U1(q, a, b); // 내장 NOR 게이트 인스턴스 생성

endmodule
```

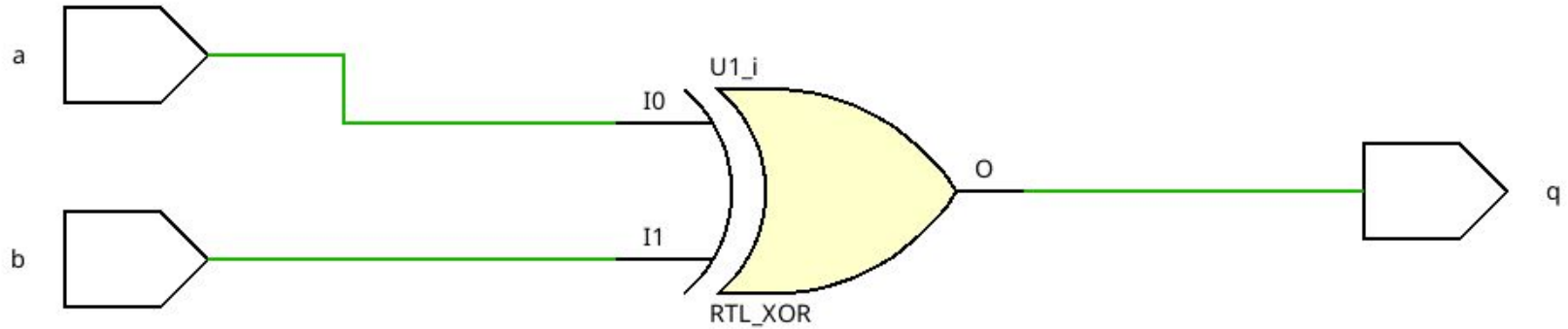
```
module nor_gate_dataflow (
    input a, b,          // 입력 신호 a, b
    output q              // 출력 신호 q (wire 타입)
);
    assign q = ~(a | b); // OR 연산 후 반전하여 NOR 구현
endmodule
```

NOR Gate (Simulation)



```
# run 1000ns
Time      a b | q
0         0 0 | 1
10000     0 1 | 0
20000     1 0 | 0
30000     1 1 | 0
```

XOR Gate (Schematic)



XOR Gate (Verilog)

```
module xor_gate_behavioral (
    input a, b,          // 입력 신호 a, b
    output reg q         // 출력 신호 q (레지스터 타입)
);
    always @(a,b) begin // a 또는 b 신호가 변경될 때 실행
        if (a == 1'b0 && b == 1'b0) // 둘 다 0일 때
            q = 1'b0;              // 출력 0 (같은 값)
        else if (a == 1'b1 && b == 1'b1) // 둘 다 1일 때
            q = 1'b0;              // 출력 0 (같은 값)
        else // 하나는 0, 하나는 1일 때
            q = 1'b1;              // 출력 1 (다른 값)
        end
    end
endmodule
```

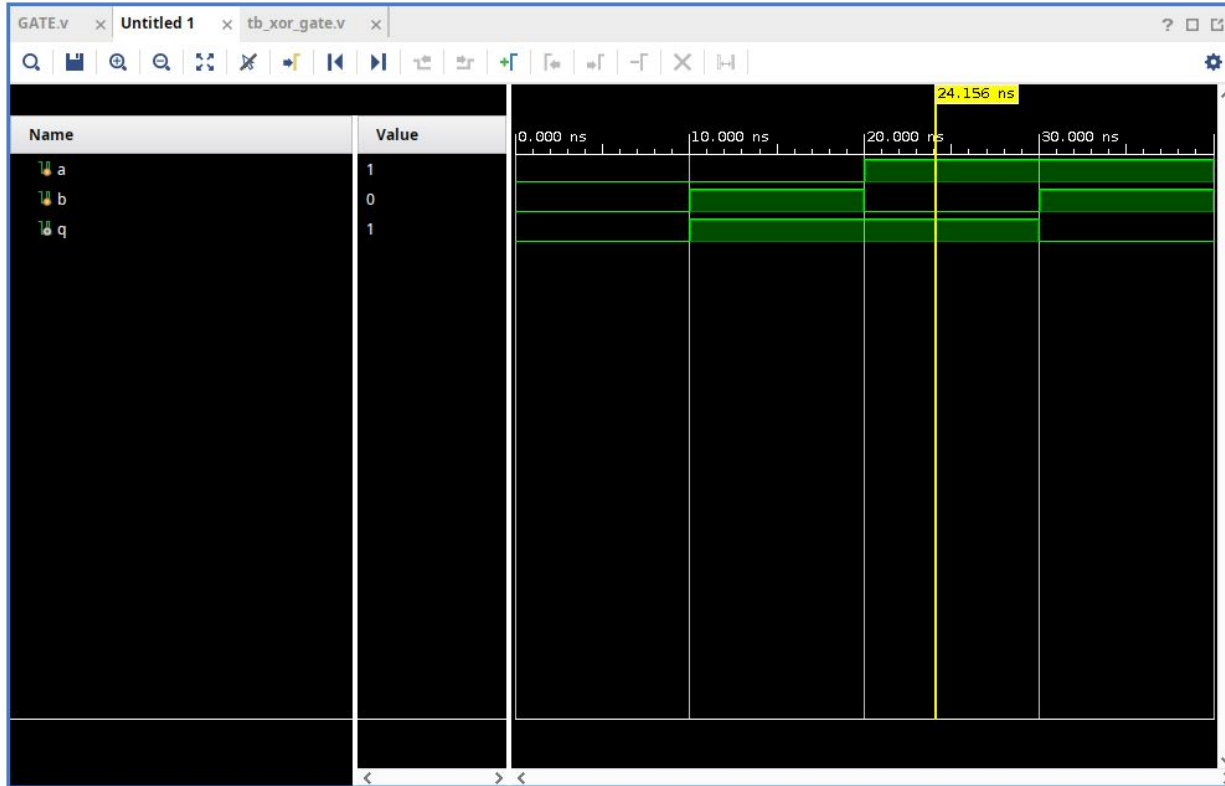
```
module xor_gate_structural(
    input a, b,          // 입력 신호 a, b
    output q             // 출력 신호 q (wire 타입)
);

    xor U1(q, a, b);     // 내장 XOR 게이트 인스턴스 생성

endmodule
```

```
module xor_gate_dataflow (
    input a, b,          // 입력 신호 a, b
    output q             // 출력 신호 q (wire 타입)
);
    assign q = a ^ b;    // XOR 연산자를 사용하여 논리 구현
endmodule
```

XOR Gate (Simulation)



```
# run 1000ns
Time    a b | q
0      0 0 | 0
10000   0 1 | 1
20000   1 0 | 1
30000   1 1 | 0
$finish called at time : 40 ns : File '
```

XNOR Gate (Schematic)



XNOR Gate (Verilog)

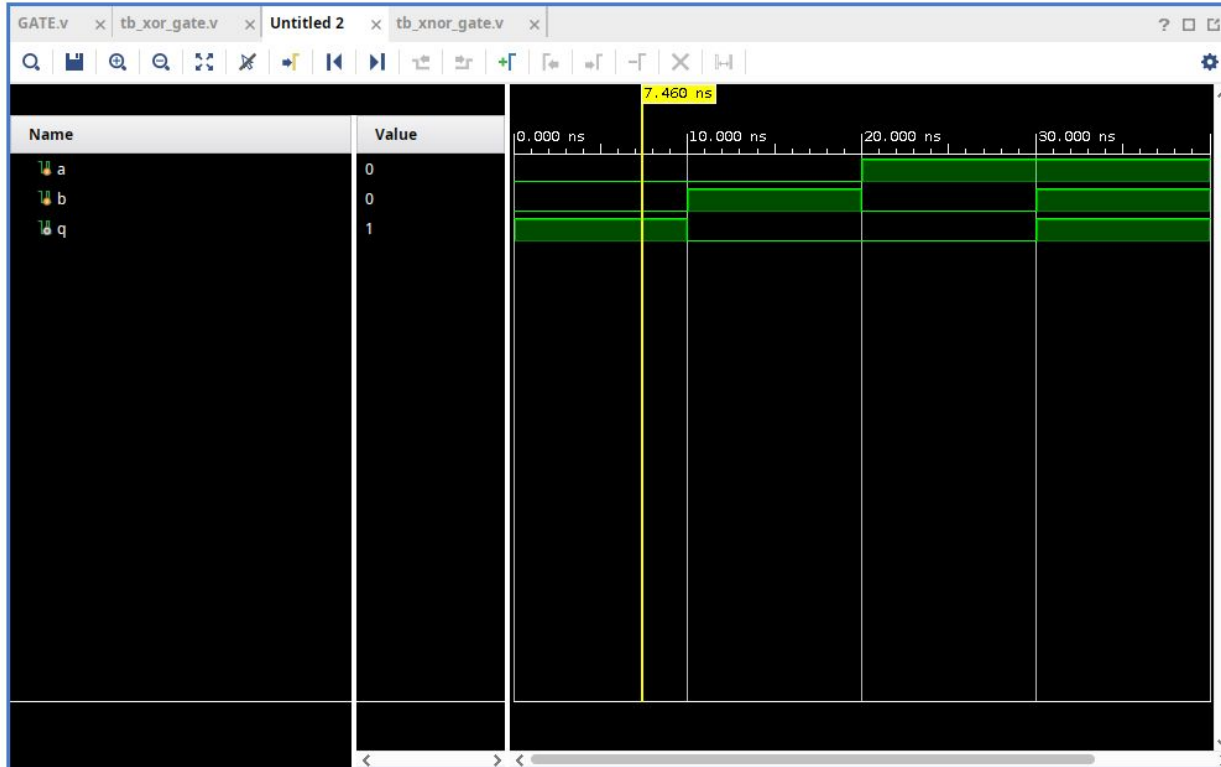
```
module xnor_gate_behavioral (
    input a, b,          // 입력 신호 a, b
    output reg q          // 출력 신호 q (레지스터 타입)
);
    always @(a,b) begin // a 또는 b 신호가 변경될 때 실행
        if (a == 1'b0 && b == 1'b0) // 둘 다 0일 때
            q = 1'b1;                // 출력 1 (같은 값)
        else if (a == 1'b1 && b == 1'b1) // 둘 다 1일 때
            q = 1'b1;                // 출력 1 (같은 값)
        else
            q = 1'b0;                // 하나는 0, 하나는 1일 때
            // 출력 0 (다른 값)
        end
    end
endmodule
```

```
module xnor_gate_structural(
    input a, b,          // 입력 신호 a, b
    output q              // 출력 신호 q (wire 타입)
);

    xnor U1(q, a, b);    // 내장 XNOR 게이트 인스턴스 생성
endmodule
```

```
module xnor_gate_dataflow (
    input a, b,          // 입력 신호 a, b
    output q              // 출력 신호 q (wire 타입)
);
    assign q = ~(a ^ b); // XOR 연산 후 반전하여 XNOR 구현
endmodule
```

XNOR Gate (Simulation)



```
# run 1000ns
Time    a b | q
0    0 0 | 1
10000   0 1 | 0
20000   1 0 | 0
30000   1 1 | 1
$finish called at time : 40 ns : 1
```